

Final Project Description

Laboratory project name:

MarketView

Developed by : Iheb Hidouri

GROUP

Laboratory group information.

Group Name:	MarketView
Members:	Iheb Hidouri
GitHub Repo Link:	https://github.com/hidouri-iheb/dev-mob

DATES

Key document dates.

Milestone	Date
Project proposal submission	08/05/2026
Project proposal acceptance	11/05/2026

BRIEF DESCRIPTION

MarketView is a native Android stock trading simulator built with Kotlin, Jetpack Compose, and MVVM that lets users track 46 real stock symbols, manage portfolios, execute simulated trades with automatic TP/SL, and compete on a global leaderboard .

DEFINITIONS

Definition of terms, abbreviations, and acronyms used.

Term	Definition
Portfolio	A collection of trades grouped under a name and strategy, with a simulated starting capital
Trade	A BUY or SELL position on a stock symbol, with entry price, size, and optional TP/SL
P&L	Profit and Loss – the financial result of a trade, calculated as the difference between entry and current/exit price multiplied by position size
Take Profit (TP)	A price threshold that automatically closes a trade when reached, locking in profits
Stop Loss (SL)	A price threshold that automatically closes a trade to limit losses
Watchlist	A personal list of stocks the user monitors, with live price updates. Three default stocks (AAPL, MSFT, GOOGL) are always displayed

Leaderboard	A global ranking of all users by portfolio return percentage
Volatile Asset	A stock with high price variability – the app ranks stocks by volatility on the home screen
Gaussian Distribution	Bell-curve probability distribution used to simulate realistic price movements
Simulated Capital	Virtual money (\$5k–\$100k) assigned to a portfolio for risk-free trading practice
Entry/Exit Price	The price at which a trade is opened (entry) and closed (exit)
BUY/SELL Position	BUY profits when price goes up; SELL (short) profits when price goes down
Directional Stocks	Two special stocks (MSTR, LCID) with forced bullish/bearish trends for testing TP/SL triggers

1. Project Context

1.1. Current Situation

Financial markets have seen extreme volatility in recent years, drawing millions of new retail investors into trading. Many entered with little knowledge and lost significant capital learning by trial and error — a pattern that created clear demand for safe environments where users can practice trading mechanics and test strategies without real money at stake.

Existing apps fail to fill this gap. Involio (now Invo), a social investing platform with over 100,000 App Store downloads, pivoted from education to real-money trading with copy-trading features, and user reviews report scammers, a ranking system that rewards gambling over skill, and payment issues. TradingView offers excellent charting tools but assumes the user already knows how to trade. The landscape splits into two extremes: apps that handle real money (dangerous for learners) or apps too simplified to teach anything meaningful.

MarketView was built to fill this gap: a realistic trading simulator that follows real stock symbols, produces believable price behavior, and lets users compete on strategy quality — all with zero financial risk.

1.2. Benefits and Value Creation

Safe learning environment – users trade with simulated capital (\$5,000 to \$100,000) with zero financial risk.

Smart navigation flows – portfolio creation lands inside the new portfolio; adding a stock navigates to watchlist; tapping an open trade navigates to its portfolio detail.

Realistic market simulation – 46 real stock symbols with Gaussian-based price movement and amplified volatility. Two directional stocks (MSTR bullish, LCID bearish) for TP/SL testing.

Competitive engagement – global leaderboard ranks all users by portfolio return percentage.

Architectural depth – complete MVVM implementation with Jetpack Compose, Room, DataStore, Firebase Auth, Coroutines, and Flow.

1.3. Project Objectives

Feature	Description
Authentication	Firestore Auth sign-up/login with username uniqueness validation via Realtime Database
Portfolio management	Create, view, delete portfolios with configurable starting balance (\$5k–\$100k) and strategy label. Creating navigates directly to the new portfolio
Simulated trading	Open BUY/SELL trades on real stock symbols with size, optional take-profit and stop-loss
Price engine	46 real stock symbols with Gaussian-based simulation ($\pm 15\%$ per session, re-baselined so sustained trends compound over time). Two directional stocks for TP/SL testing.
Automatic TP/SL	Trades auto-close when take-profit or stop-loss thresholds are hit, with P&L realized to portfolio
Real-time P&L	Unrealized P&L updates live on open trades; realized P&L persisted on close; live balance and P&L that update at the price-tick cadence .
Global leaderboard	Cross-user ranking by return %
Watchlist	Search 46 stocks, view detail card with market data, add to personal watchlist. Three default stocks (AAPL, MSFT, GOOGL) always visible. Adding a stock navigates to watchlist
Trade history	Complete log of closed trades with entry/exit prices, open/close timestamps, and P&L
Theme system	Light/dark toggle persisted via DataStore with centralized MaterialTheme
Confirmation UX	Every destructive action requires explicit confirmation dialog
Snackbar feedback	Every successful action produces a visible notification
Animations	Screen transitions (fade/slide), AnimatedVisibility, shimmer loading, swipe-to-delete

Contextual navigation	Open trades on HomeScreen navigate to portfolio detail; portfolio creation lands inside the new portfolio
-----------------------	---

2. Project Profile

2.1. Project Scope

Scope	Details
What was developed	Complete trading simulator: auth, portfolios, trades, watchlist, leaderboard, settings, trade history, theme, animations, confirmation/feedback UX, contextual navigation flows
External services	Firestore Authentication (login/signup) + Firestore Realtime Database (username uniqueness)
Local storage	Room for portfolios, trades, watchlist – DataStore for theme preferences
Price engine	FakeStockRepository (Gaussian simulation, 46 stocks, 500ms updates, amplified volatility) as fallback for API rate limits. RealStockRepository (Retrofit + Twelve Data API) is implemented and ready
<u>Future development</u>	Leverage on trades , Price & performance predictions using machine learning , user profile pages, interactive price charts, real-time API activation with paid plan, push notifications for TP/SL triggers, Price & performance predictions

2.2. Solution Profile

The app follows MVVM with four layers:

Layer	Technology	Responsibility
-------	------------	----------------

UI	Jetpack Compose	Declarative screens, NavHost + bottom bar, animations, screen transitions, state observation via collectAsStateWithLifecycle
ViewModel	StateFlow + viewModelScope	Screen state management, reactive data merging via combine on shared StateFlows (stateIn), user action handling
Repository	Kotlin interfaces	Abstracts data sources. FakeStockRepository and RealStockRepository share one interface – swappable in one line
Data	Room + DataStore + Firebase	Room for structured relational data (3 entities, 3 DAOs), DataStore for preferences, Firebase for auth

Business logic is extracted into independent classes: for example : PnLCalculator (profit/loss computation, portfolio summaries, auto-close detection) and SignUpValidator . These classes have no Android dependencies and are fully unit-testable.

3. Constraints and Assumptions (Requirements)

3.1. Technical Constraints

Functional Requirements

Requirement	Implementation
User authentication (signup, login, logout)	Firebase Auth + Realtime Database for username uniqueness
Portfolio CRUD operations	Room entities with PortfolioDao, cascade delete on related trades
Open/close trades with live P&L	Trade entity, combine(portfolios, trades, prices) in ViewModel
Automatic take-profit / stop-loss	PnLCalculator.shouldAutoClose() checked once per trade per price tick in a single-pass loop
Stock search and watchlist	SearchableStocks local catalog + WatchedStock Room entity + 3 default stocks always visible for better UX .
Global leaderboard	PortfolioDao.getAllGlobal() + PnLCalculator.buildSummaries()
Trade history with timestamps	Trade.openedAt / closedAt fields, formatted with SimpleDateFormat
Confirmation before destructive actions	Reusable ConfirmDialog composable used across 4 screens
Feedback on successful actions	SnackbarHostState in Scaffold, triggered after each action
Contextual navigation	Portfolio creation → detail screen; add to watchlist → watchlist tab; open trade tap → portfolio detail

Non-Functional Requirements

Requirement	How Achieved
Real-time price updates	Flow emitting every 500ms, collected via collectAsStateWithLifecycle. Upstream flows shared with stateIn(WhileSubscribed(5000))
Data persistence across restarts	Room + DataStore – all data survives app and process death
Consistent UI across all screens	MaterialTheme with centralized colors, typography, 8dp spacing grid, 16dp card radius

State survives configuration changes	ViewModel + StateFlow – state is preserved on rotation or recreation
Smooth user experience	Fade/slide screen transitions, AnimatedVisibility, shimmer loading, animated balance/P&L counters (2s tween)
Swappable data source	StockRepository interface – switch from simulation to real API by changing one line in MarketViewApplication.kt

3.2. Assumptions

The app is designed to use the Twelve Data API for real stock prices. Due to API rate limits on the free tier (8 requests/minute for 46 stocks), a local Gaussian simulation engine was implemented as a development and testing fallback. Both implementations share the same StockRepository interface.

The simulation engine uses real stock symbols and realistic base prices, so the trading experience closely mirrors real market conditions.

Two stocks (MSTR, LCID) have been forced directional trends to facilitate testing of take-profit and stop-loss auto-close logic , and leaderboard overlaps .

4. Design

4.1. MVVM Architecture

The entire application follows the Model-View-ViewModel pattern with strict separation:

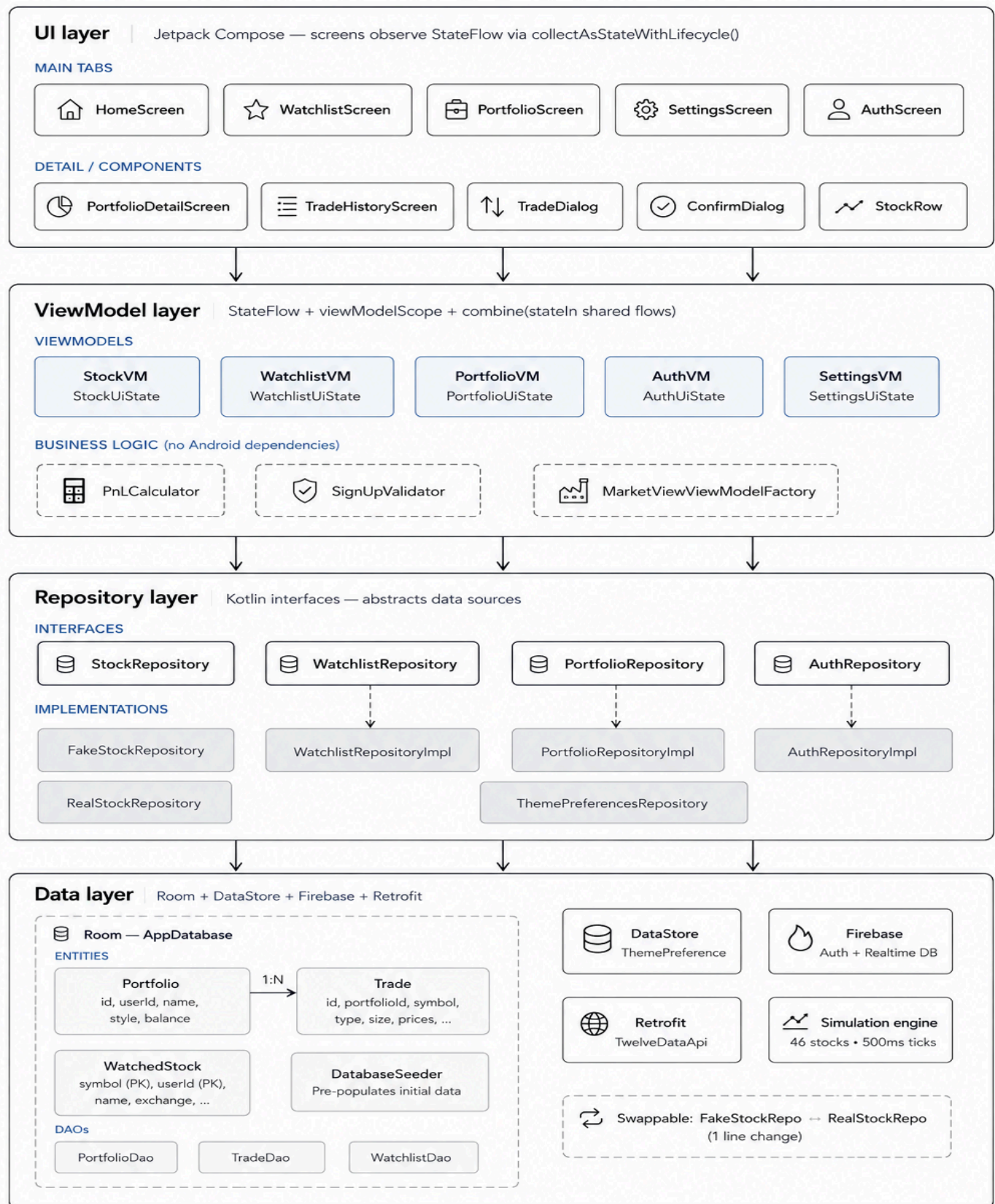
View (UI) – Composable functions in ui/screens/. They receive state as parameters and emit events as lambda callbacks. They never import a repository or a DAO.

ViewModel – Classes in viewmodels/. They hold MutableStateFlow, collect from repositories using viewModelScope.launch, transform data into screen-ready UiState objects, and expose them as StateFlow.

Model (Data) – Room entities, DAOs, and repositories. The ViewModel never touches a DAO directly – it goes through the repository interface.

MVVM Architecture Diagram:

MarketView — MVVM architecture



Example: how portfolio balance reaches the screen:

```
// 1. DAO emits Flow<List<Portfolio>>
@Query("SELECT * FROM portfolios WHERE userId = :userId")
fun getAll(userId: String): Flow<List<Portfolio>>

// 2. Repository wraps it behind an interface
override fun getAllPortfolios(userId: String) = portfolioDao.getAll(userId)

// 3. ViewModel shares upstream flows with stateIn, then combines them
private val portfoliosFlow = portfolioRepository.getAllPortfolios(userId)
    .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

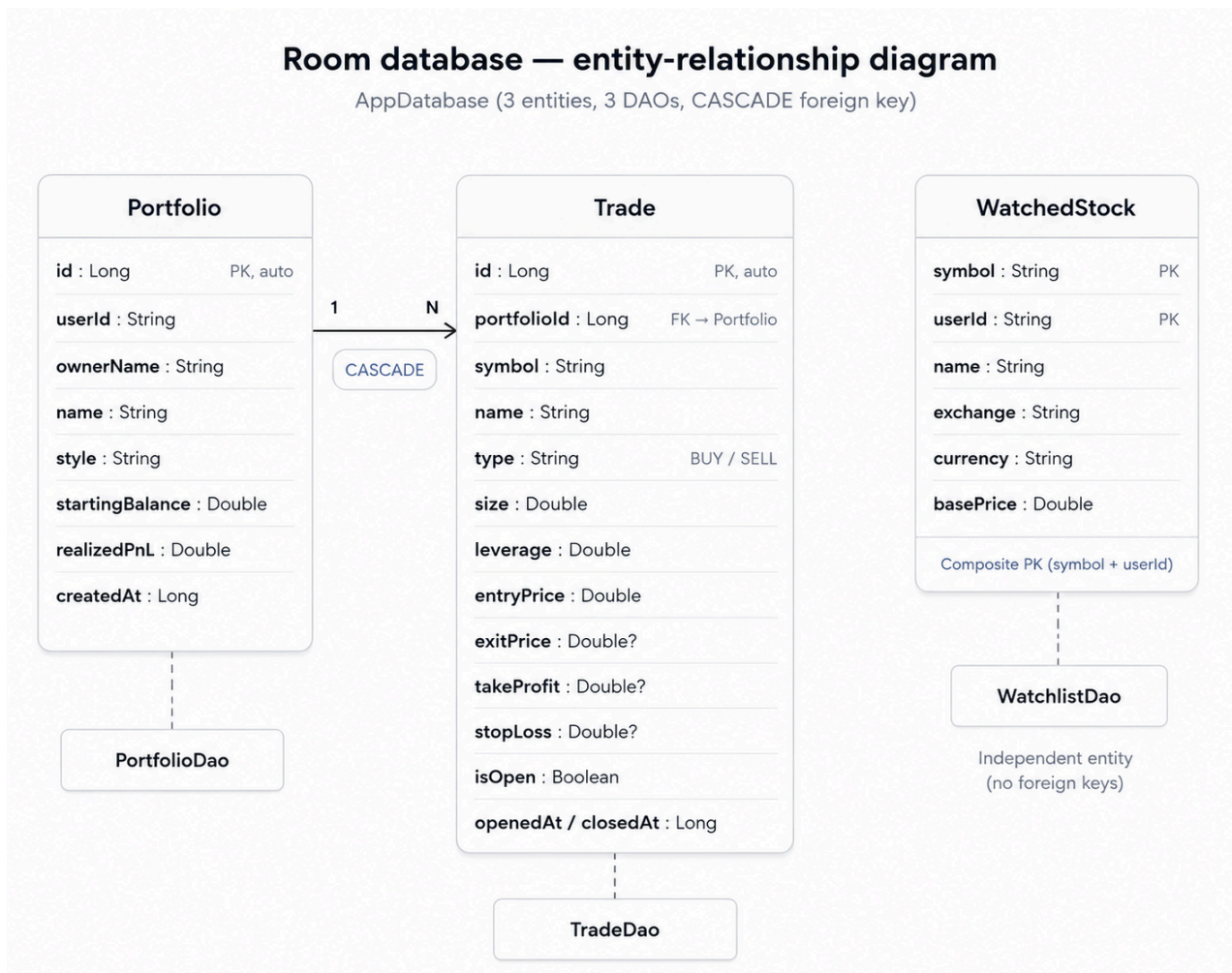
combine(portfoliosFlow, allTradesFlow, stocksFlow) { p, t, s ->
    pnlCalculator.buildSummaries(p, t, LivePrices)
}.collect { _listState.value = PortfolioListUiState(it) }

// 4. UI observes with Lifecycle awareness
val state by viewModel.listState.collectAsStateWithLifecycle()
```

4.2. Room Database Design

Entity	Primary Key	Key Fields	Relations
Portfolio	id (autoGenerate)	userId, ownerName, name, style, startingBalance, realizedPnL, createdAt	Has many Trades (portfolioId FK)
Trade	id (autoGenerate)	portfolioId, symbol, name, type, size, entryPrice, exitPrice, TP, SL, isOpen, openedAt, closedAt	Belongs to Portfolio (CASCADE delete)
WatchedStock	composite (symbol + userId)	name, exchange, currency, basePrice	Independent

Entity-Relationship Diagram:



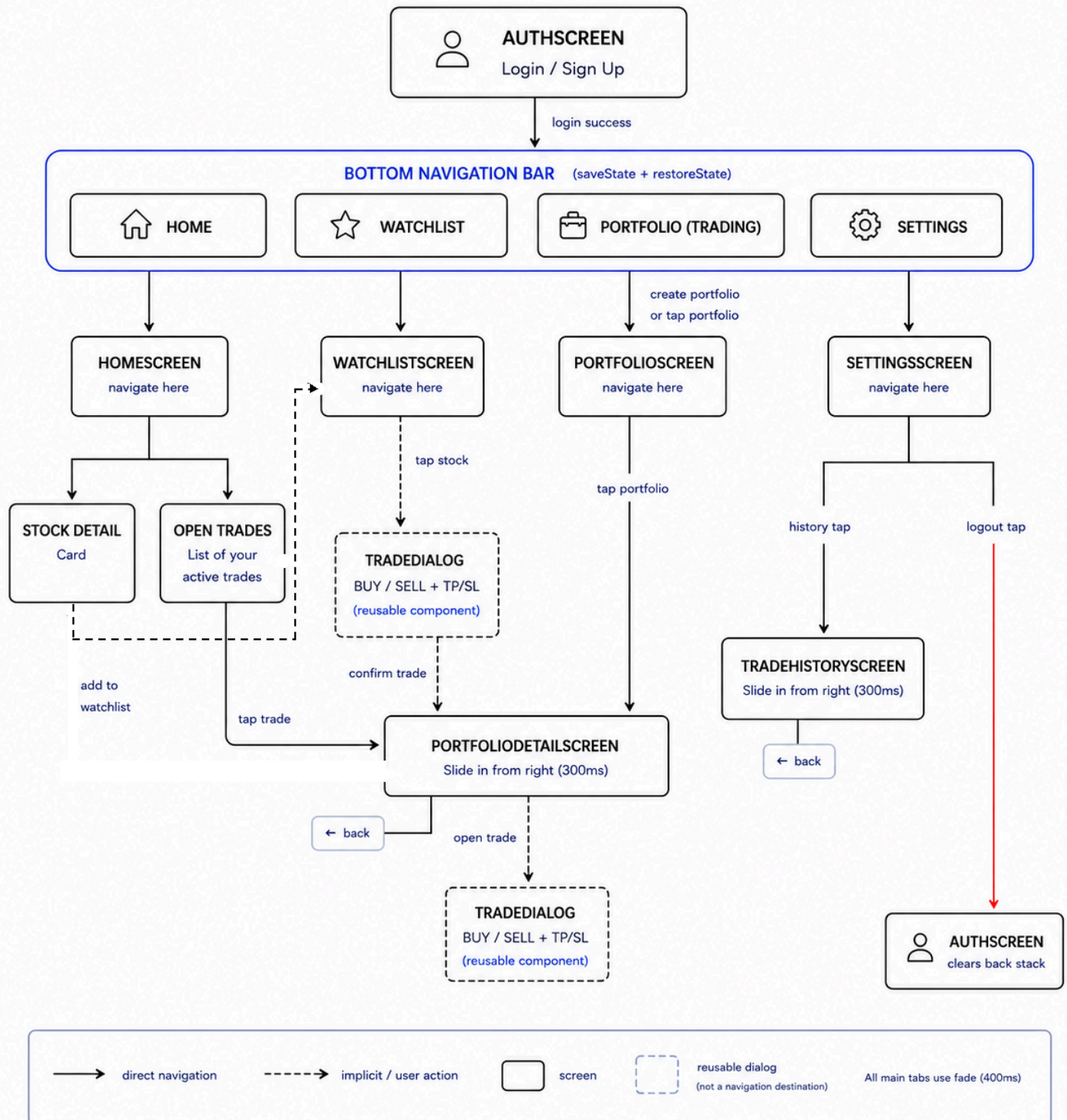
Deleting a portfolio automatically removes all its trades through the CASCADE foreign key. This was chosen over manual cleanup to guarantee referential integrity at the database level.

4.3. Navigation

Route	Screen	Type	Transition
home	HomeScreen	Main tab	Fade in (400ms)
watchlist	WatchlistScreen	Main tab	Fade in (400ms)
portfolio	PortfolioScreen (Trading)	Main tab	Fade in (400ms)
settings	SettingsScreen	Main tab	Fade in (400ms)
portfolio_detail	PortfolioDetailScreen	Detail	Slide in from right (300ms)
trade_history	TradeHistoryScreen	Detail	Slide in from right (300ms)

Navigation Graph Diagram:

MarketView — Navigation Graph



Detail screens use slide transitions with `popEnterTransition/popExitTransition` for natural forward-back navigation.

4.4. State Management

Each screen has a dedicated `UiState` data class, stored in its own file alongside the corresponding `ViewModel`:

UiState	ViewModel	Key Fields
AuthUiState	AuthViewModel	isLoggedIn, isLoading, error, displayName, field-level errors
StockUiState	StockViewModel	stocks, trending (sorted by volatility), isLoading, error
WatchlistUiState	WatchlistViewModel	stocks (merged: saved symbols + live prices + 3 defaults)
PortfolioListUiState	PortfolioViewModel	portfolios (list of PortfolioSummary with P&L)
PortfolioDetailUiState	PortfolioViewModel	portfolio, trades with live P&L, totalPnL, currentBalance
SettingsUiState	SettingsViewModel	themeMode (LIGHT / DARK)

All upstream data flows (portfolios, trades, stock prices) are shared using `stateIn(WhileSubscribed(5000))` so that each data source is collected once and the result is reused across multiple combine blocks. This avoids redundant database listeners and improves efficiency.

4.5. Repository Interface Pattern

The `StockRepository` interface is the clearest example of why abstraction matters in this project:

```
interface StockRepository {  
    fun getStocks(): Flow<List<Stock>>  
    suspend fun getStockDetail(symbol: String): Stock?  
}
```

```
// Implementation 1: Local Gaussian simulation (active)  
class FakeStockRepository : StockRepository { ... }
```

```
// Implementation 2: Twelve Data API via Retrofit (ready)  
class RealStockRepository : StockRepository { ... }
```

In `MarketViewApplication.kt`, switching from simulation to real data requires changing one line:

```
val stockRepository: StockRepository by lazy {
```

```
FakeStockRepository(applicationScope) // → RealStockRepository() }
```

5. Implementation

5.1. Project Structure

```
com.ihebhidouri.marketview/  
├── data/  
│   ├── room/entity/      Portfolio.kt, Trade.kt, WatchedStock.kt  
│   ├── room/dao/         PortfolioDao.kt, TradeDao.kt, WatchlistDao.kt  
│   ├── room/database/    AppDatabase.kt, DatabaseSeeder.kt  
│   ├── datastore/        ThemePreference.kt  
│   ├── remote/           RetrofitClient.kt, TwelveDataApi.kt, Dto.kt  
│   └── SearchableStocks.kt  
├── models/                Stock.kt, PortfolioSummary.kt, PnLCalculator.kt,  
                           SignUpValidator.kt, ThemeMode.kt  
├── repository/            Interfaces + Implementations (5 repos)  
├── viewmodels/            5 ViewModels + 5 UiState files + Factory  
├── ui/  
│   ├── navigation/       AppNavGraph.kt, Routes.kt, BottomNavItem.kt  
│   ├── screens/          7 Screens ( home , watchlist , portfolio , portfolio detail .. )  
│   ├── components/       ConfirmDialog.kt, StockRow.kt, TradeDialog.kt, UserAvatar.kt  
│   └── theme/             Color.kt, Type.kt, Theme.kt  
├── MainActivity.kt  
└── Marketviewapplication.kt
```

5.2. Price Simulation Engine

The FakeStockRepository simulates 46 real stock symbols using `java.util.Random.nextGaussian()`. This method was chosen specifically because it produces a bell-curve distribution – `kotlin.random.Random` does not offer an equivalent function. The result is natural-looking price behavior with amplified volatility:

```
val forcedTrend = when (seed.symbol) {  
    "MSTR" -> 0.02 // always bullish  
    "LCID" -> -0.02 // always bearish  
    else -> 0.0  
}  
  
val newTrend = state.trend * 0.95 + random.nextGaussian() * 0.005 + forcedTrend  
val nudge = newTrend + random.nextGaussian() * 0.08  
val newChangePercent = (state.changePercent + nudge).coerceIn(-15.0, 15.0)
```

Prices are shared across all screens using `stateIn(WhileSubscribed(5000))`, so the simulation only runs when at least one screen is actively collecting. This avoids wasting CPU when the app is in the background.

5.3. P&L Calculation

PnLCalculator is a standalone class:

```
fun calculate(trade: Trade, currentPrice: Double): Double {  
    val diff = if (trade.type == "BUY")  
        currentPrice - trade.entryPrice  
    else trade.entryPrice - currentPrice  
    return diff * trade.size * trade.Leverage  
}
```

It also handles auto-close logic: on every price tick, `shouldAutoClose()` checks if the current price has crossed the trade's take-profit or stop-loss threshold. This is done in a single-pass loop – each trade is checked once, and trades that need closing are processed immediately while remaining trades are collected into the UI state.

5.4. Manual Dependency Injection

Without a DI framework, all dependencies are managed manually through `MarketViewViewModelFactory`. All repositories are lazily initialized in `MarketViewApplication` using Kotlin's by lazy delegate, ensuring they are created only when first accessed.

5.5. Theme and UI System

All colors are defined in `Color.kt`, typography in `Type.kt`, and the scheme in `Theme.kt`. Every composable uses `MaterialTheme.colorScheme` and `MaterialTheme.typography` exclusively – no hardcoded colors or sizes. The theme supports light and dark modes, with the preference persisted via `DataStore`. Spacing follows an 8dp grid throughout (8, 16, 24, 32dp). All cards use a consistent 16dp corner radius. Balance and P&L update live in step with the 500ms price stream, shown as clean discrete values rather than a per-frame animated counter, so the figures stay readable as prices move.

6. Testing

The testing strategy covers three layers of the architecture:

Test Layer	File	# Tests	Coverage
Unit (JUnit)	PnLCalculatorTest	18	BUY/SELL P&L, leverage, closed trades, portfolio summaries, auto-close TP/SL

Unit (JUnit)	SignUpValidatorTest	17	Username rules, email format (Kotlin regex), password complexity, combined validation
Unit (JUnit)	StockViewModelTest	4	Initial state, stock loading, trending sort, error handling
Unit (JUnit)	PortfolioViewModelTest	3	Portfolio creation, selection, state updates
Unit (JUnit)	SettingsViewModelTest	1	Theme mode toggle
Unit (JUnit)	RealStockRepositoryTest	1	API response mapping
Instrumented	WatchlistDaoTest	1	Room insert, query, Flow observation
UI (Compose)	AuthScreenTest	5	Login form render, button states, signup toggle, error display
UI (Compose)	ConfirmDialogTest	4	Content display, confirm/cancel callbacks, default labels
UI (Compose)	SettingsScreenTest	4	Profile display, theme options, theme change callback, logout dialog
UI (Compose)	WatchlistScreenTest	3	Empty state, populated stock list, subtitle display
UI (Compose)	TradeDialogTest	4	Title display, no-portfolio error, BUY/SELL options, pre-selected stock
UI (Compose)	HomeScreenTest	3	Leaderboard ranking, open trades display, error state with retry

Why these tests

PnLCalculator is the financial core – if P&L is wrong, the entire app’s purpose breaks. Tested exhaustively with 18 cases.

SignUpValidator gates user entry – must correctly accept valid inputs and reject invalid ones. Email validation uses pure Kotlin regex (not android.util.Patterns) so it can be tested without Android framework dependencies.

WatchlistDao verifies the data layer works correctly in isolation on an actual Android device.

AuthScreen is the first thing every user sees – critical path.

ConfirmDialog is reused across 4 screens – a single bug would affect delete, close, remove, and logout.

HomeScreenTest validates the leaderboard ranking and open trades display – the two most data-heavy sections of the app.

SettingsScreenTest covers the theme toggle – a DataStore integration point that the course emphasizes.

TradeDialogTest validates the trade creation flow including error states and portfolio dependency.

Testing methodology

Beyond automated tests, I tested the app manually by creating multiple user accounts and simulating different trading scenarios – portfolios with varying strategies, trades with different entry points, and deliberate TP/SL triggers using the directional stocks (MSTR for bullish, LCID for bearish). The DatabaseSeeder was also used to generate pre-populated profiles with different performance outcomes, allowing me to verify the leaderboard ranking logic, P&L calculations across accounts, and the overall app behavior with a populated database.

7. Conclusions

MarketView demonstrates the application of the core course concepts in a real, cohesive project:

Concept	Application in MarketView
MVVM	Strict separation: composables receive state, ViewModels manage flows, repositories abstract data
Jetpack Compose	Entire UI is declarative – 7screen files, no XML layouts, state hoisting throughout
Room	3 entities, 3 DAOs, foreign keys with cascade delete, properly structured in data/room/
DataStore	Theme preference as key-value – the right tool for simple preferences
Coroutines	viewModelScope.launch for all async operations, no main-thread blocking
Flow + combine	Price streams, portfolios, and trades merged reactively. Upstream flows shared with stateIn – UI auto-updates
StateFlow	Every ViewModel exposes StateFlow, collected with collectAsStateWithLifecycle
Navigation	NavHost with 6 routes, bottom bar, transitions, popUpTo with saveState/restoreState, contextual navigation
Scaffold	Bottom bar, SnackbarHost, proper content padding
State hoisting	Composables receive state and callbacks – no internal data management
Repository pattern	Interface with two implementations – swap data source without touching UI
Testing	68 tests across unit, instrumented, and UI layers covering all architecture levels

Every requirement is implemented and verified. The automated tests confirm the correctness of business logic, database operations, and UI behavior. Manual testing across multiple accounts validated the end-to-end trading flow and leaderboard ranking.